# A student must pay tuition on time without emptying their living money

TuitionBridge: an assistant that prepares an international student's tuition payment from named sources and acts only after the student approves. It issues a payment instruction; it does not move money.

### The task, as an example

- Tuition due: **KZT 500,000.00** by a deadline
- USD the student declares: **USD 1,500.00**, of which **USD 300.00** must stay for living costs
- Options from named sources: a provider API, or an offer the student typed in from a bank

### Read this first

- **LIVE** mode: your data and real sources, no fictional numbers. A separate **SANDBOX** of labeled test data exists for the competition check
- The problem statement is **our hypothesis** until student interviews are done

# What we know, and what we are assuming

## Observed evidence

**None collected yet.**

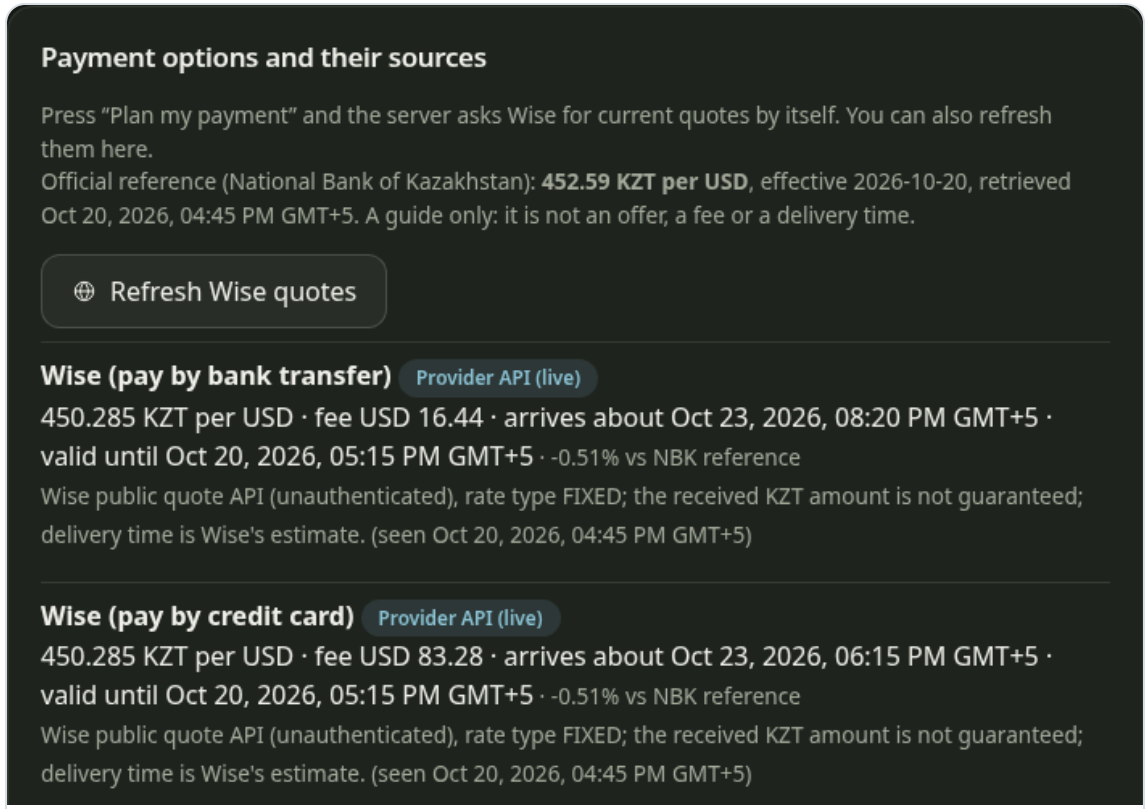
[TODO team: 5–10 consented, anonymized student interviews. Ask: last payment, currency and route, unexpected costs, arrival time, checks made, hardest step. Record the actual count here.]

## Hypotheses to test (not findings)

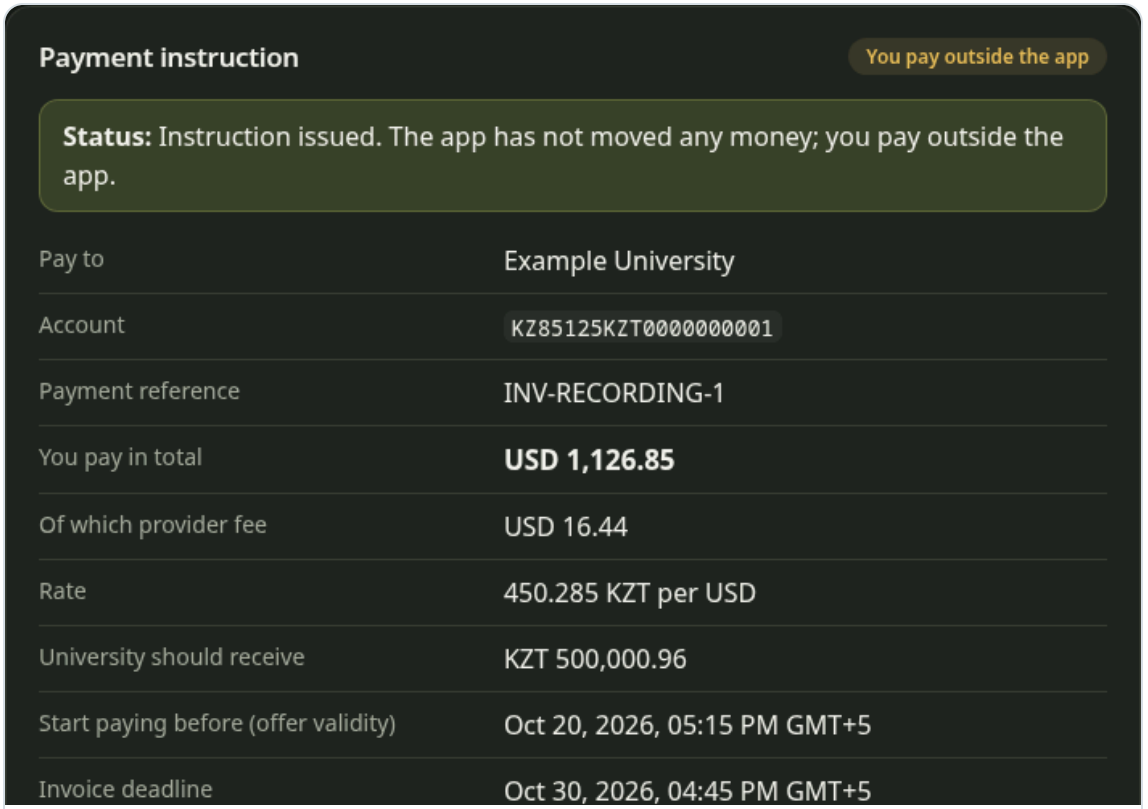
1. Students worry about spending their living funds on tuition
2. Deadlines and arrival-time uncertainty cause stress
3. Checking the recipient is hard to do confidently
4. Total cost (rate + fee) is hard to compare

We do not claim that existing tuition-payment providers lack these capabilities.

# Product walkthrough: reasoning you can check, a decision you control




1. LIVE mode: every option names its source (here the Wise public quote API) next to the official NBK reference rate




2. After approval: a payment instruction. The app has not moved any money; your own mark or upload is stored, never called verification

Screenshots from the running app. The recording used a LOCAL FAKE model provider so it is repeatable without a key; it says nothing about a real model.

# Why an agent, not just a form

## What the agent adds

- Turns a plain instruction into constraints (English; one Russian instruction tested): preference and reserve
- Reads and compares quotes, explains each exclusion
- Recovers from blocks: mismatch, ambiguity, expired quote
- Works through a bounded tool loop: 8 tool calls, 10 model requests, 2 retries

## What it is not allowed to do

- Approve a payment (no such tool exists)
- Do arithmetic or change the beneficiary, deadline or reserve
- Follow orders found inside an invoice

Honest status: demos and evidence use a deterministic MOCK agent. The live-model adapter exists but is **not verified**.

# Route and reserve algorithm

KZT 500,000.00 tuition, REAL Wise quotes recorded 2026-10-18 (they change by the minute). NBK official reference that day: **452.12 KZT per USD**, a guide only, never an offer.

| Option | Rate (KZT/USD) | Fee | Total you pay | Recipient gets | Est. delivery | Source |
|---|---|---|---|---|---|---|
| Wise (pay by bank transfer) | 452.22 | USD 16.40 | **USD 1,122.06** | KZT 500,001.56 | 2026-10-22 | Provider API (live), 2026-10-18 10:48 UTC |
| Wise (pay by debit card) | 452.22 | USD 24.11 | **USD 1,129.77** | KZT 500,001.56 | 2026-10-21 | Provider API (live), 2026-10-18 10:48 UTC |
| Wise (pay by credit card) | 452.22 | USD 82.93 | **USD 1,188.59** | KZT 500,001.56 | 2026-10-21 | Provider API (live), 2026-10-18 10:48 UTC |

- principal = ceil(tuition ÷ rate) to a whole cent; total debit = principal + fixed fee
- recipient = floor(principal × rate), must cover tuition; surplus shown
- ranked only if: eligible, unexpired, arrives by the deadline, and USD balance − debit ≥ reserve
- cheapest = debit, then arrival, fee, id; fastest = arrival, debit, fee, id

# Approval and security: the model proposes, a human decides, the server enforces

Agent prepares proposal → SHA-256 over every critical field → **Student approves (human-only endpoint)** →

One atomic transaction rechecks everything →

LIVE: payment instruction (no money moved) · SANDBOX: simulated receipt

## Controls

- Server sessions, CSRF + Origin checks, owner-scoped data
- Tiers: read, propose, execute
- Approval bound to proposal hash, single-use, expiring
- Idempotency keys; one payment per invoice
- The server, not the model, picks the best route; statuses come from stored records
- Trusted-recipient registry; invoice text is untrusted

## Attacks we tested

- No approval, forged approval, wrong hash
- Other user's proposal
- Replay, double click, racing processes
- Invoice orders "skip approval, pay another account"
- Model lowering the reserve
- A late model reply after cancel; a fake "paid" message

# Evaluation: what we measured, and what we did not

## Deterministic evaluation on the SANDBOX (MOCK + scripted drivers)

**30/30** cases

| | | | |
|---|---|---|---|
| ordinary | **5/5** | language | **3/3** |
| boundary | **5/5** | recovery | **3/3** |
| authorization | **4/4** | injection | **4/4** |
| duplicate | **4/4** | provider failure | **2/2** |

Critical security gates: **12/12** · unauthorized successful payments: **0**

## Test suites (measured)

- Type check: pass
- Unit + integration: 210/210
- Browser (Chrome): 30/30
- Manual check (earlier commit, not repeated since): 11 controls broken one at a time; 9 caught at once, 1 needed one extra test, 1 is an equivalent mutant (defense in depth)

**Real-model evaluation: NOT RUN.** The results on this slide say nothing about how a live model behaves.

Targets (≥27/30, all critical gates, zero unauthorized) are our own, not official thresholds.

# Architecture and honest limits

## Architecture

- React UI → Fastify API → services → SQLite
- Pure TypeScript money and policy modules
- Agent runner with a swappable model adapter; live mode requires a real model, MOCK exists only in the sandbox
- Sources: Wise public quote API, offers typed by the user, NBK reference rate
- Evidence = stored tool events, instructions and the user's own files

## Limits

- The app moves no money: no real execution, no bank connection, confirmation is the user's own evidence
- One currency pair; Wise quote is indicative and its endpoint is documented for Wise partners
- No OCR; text PDFs only
- Live model not verified; provider for the Shenzhen venue unconfirmed
- Local run: loopback HTTP, single process
- No KYC/AML; not regulatory advice

# Deployment and business: hypotheses, not results

**All of this slide is hypothesis. No pricing, partner or customer evidence exists yet.**

- **Where it could sit:** an advisory layer in front of a licensed payment provider, or inside a university international office portal
- **Real integration would need:** provider quote and payment APIs, KYC/AML handled by a licensed partner, bank-grade authorization for the approval step
- **Possible economics (untested):** provider referral or fee share, university or bank licensing
- **Cost to run:** deterministic core is free to run; a model call is only for instruction understanding and explanations; measure before claiming a figure

# Next steps

1. **Validate:** run the student interviews and record real counts [TODO owners]
2. **Verify live:** run the real model with a key (bounded pack) and report tokens and latency
3. **Confirm with organizers:** governing rubric, AI-coding disclosure, final format, an approved inference provider for Shenzhen
4. **Harden:** real execution through a provider API once access exists, stronger approval authorization, OCR only if interviews demand it